```
CREATE DATABASE ecommerce_analysis;
USE ecommerce_analysis;
SELECT * FROM customers;
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM order_items;
SELECT * FROM payments;
SELECT * FROM cart_activity;



CREATE TABLE customers (
   customer_id VARCHAR(20) PRIMARY KEY,
   customer_name VARCHAR(100),
   gender VARCHAR(20),
   age INT,
   city VARCHAR(100),
   state VARCHAR(100),
   region VARCHAR(50),
   signup_date DATE,
   customer_segment VARCHAR(50)
);


USE ecommerce_analysis;


SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
```

```
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL
SELECT 'payments', COUNT(*) FROM payments
UNION ALL
SELECT 'cart_activity', COUNT(*) FROM cart_activity;

SELECT COUNT(*) AS invalid_customer_ids
FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

SELECT COUNT(*) AS invalid_order_ids
FROM order_items oi
LEFT JOIN orders o
  ON oi.order_id = o.order_id
WHERE o.order_id IS NULL;

SELECT COUNT(*) AS invalid_cart_records
FROM cart_activity ca
LEFT JOIN customers c
  ON ca.customer_id = c.customer_id
LEFT JOIN products p
  ON ca.product_id = p.product_id
WHERE c.customer_id IS NULL
  OR p.product_id IS NULL;
```

```
  SELECT
   ROUND(SUM(quantity * unit_price * (1 - discount_percent / 100)), 2) AS total_revenue
FROM order_items;

SELECT
   COUNT(DISTINCT order_id) AS total_orders
FROM orders
WHERE order_status = 'Delivered';

SELECT
   ROUND(
      SUM(
         oi.quantity *
         (p.selling_price - p.cost_price) *
         (1 - oi.discount_percent / 100)
      ), 2
   ) AS total_profit
FROM order_items oi
JOIN products p
   ON oi.product_id = p.product_id
JOIN orders o
   ON oi.order_id = o.order_id
WHERE o.order_status = 'Delivered';

SELECT
   ROUND(
      SUM(oi.quantity * oi.unit_price * (1 - oi.discount_percent / 100))
```

```
    / COUNT(DISTINCT o.order_id),
    2
  ) AS average_order_value
FROM order_items oi
JOIN orders o
  ON oi.order_id = o.order_id
WHERE o.order_status = 'Delivered';

SELECT
  DATE_FORMAT(o.order_date, '%Y-%m') AS month,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS monthly_revenue
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
WHERE o.order_status = 'Delivered'
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;

SELECT
  DATE_FORMAT(o.order_date, '%Y-%m') AS month,
  ROUND(
    SUM(
      oi.quantity *
```

```
        (p.selling_price - p.cost_price) *
        (1 - oi.discount_percent / 100)
      ), 2
    ) AS monthly_profit
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
WHERE o.order_status = 'Delivered'
GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;

SELECT
  p.category,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS category_revenue
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
WHERE o.order_status = 'Delivered'
GROUP BY p.category
```

```
ORDER BY category_revenue DESC;


SELECT
  p.product_id,
  p.product_name,
  p.category,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS total_revenue
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
WHERE o.order_status = 'Delivered'
GROUP BY
  p.product_id,
  p.product_name,
  p.category
ORDER BY total_revenue DESC
LIMIT 10;


SELECT
  p.product_id,
  p.product_name,
```

```
  p.category,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS total_revenue
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
WHERE o.order_status = 'Delivered'
GROUP BY
  p.product_id,
  p.product_name,
  p.category
ORDER BY total_revenue ASC
LIMIT 10;

SELECT
  o.region,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS regional_revenue
```

```
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
WHERE o.order_status = 'Delivered'
GROUP BY o.region
ORDER BY regional_revenue DESC;

SELECT
  customer_type,
  COUNT(*) AS customer_count
FROM (
  SELECT
    o.customer_id,
    CASE
      WHEN COUNT(DISTINCT o.order_id) > 1 THEN 'Repeat Customer'
      ELSE 'One-Time Customer'
    END AS customer_type
  FROM orders o
  WHERE o.order_status = 'Delivered'
  GROUP BY o.customer_id
) AS customer_orders
GROUP BY customer_type;

SELECT
  o.customer_id,
  c.customer_name,
  COUNT(DISTINCT o.order_id) AS total_orders,
  ROUND(
```

```
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
      ), 2
    ) AS total_spent
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
WHERE o.order_status = 'Delivered'
GROUP BY
  o.customer_id,
  c.customer_name
ORDER BY total_spent DESC;


SELECT
  o.customer_id,
  MAX(DATE(o.order_date)) AS last_purchase_date,
  DATEDIFF(
    (SELECT MAX(DATE(order_date))
     FROM orders
     WHERE order_status = 'Delivered'),
    MAX(DATE(o.order_date))
  ) AS recency_days
FROM orders o
WHERE o.order_status = 'Delivered'
GROUP BY o.customer_id
```

```
ORDER BY recency_days ASC;


SELECT
  customer_id,
  COUNT(DISTINCT order_id) AS frequency
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY frequency DESC;


SELECT
  o.customer_id,
  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS monetary
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
WHERE o.order_status = 'Delivered'
GROUP BY o.customer_id
ORDER BY monetary DESC;


WITH rfm AS (
  SELECT
    o.customer_id,
```

```
      DATEDIFF(
        (SELECT MAX(DATE(order_date))
         FROM orders
         WHERE order_status = 'Delivered'),
        MAX(DATE(o.order_date))
      ) AS recency,
      COUNT(DISTINCT o.order_id) AS frequency,
      ROUND(
        SUM(
          oi.quantity * oi.unit_price *
          (1 - oi.discount_percent / 100)
        ), 2
      ) AS monetary
    FROM orders o
    JOIN order_items oi
      ON o.order_id = oi.order_id
    WHERE o.order_status = 'Delivered'
    GROUP BY o.customer_id
)
SELECT
    customer_id,
    recency,
    frequency,
    monetary,
    NTILE(5) OVER (ORDER BY recency DESC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
FROM rfm;
```

```
WITH rfm AS (
  SELECT
    o.customer_id,
    DATEDIFF(
      (SELECT MAX(DATE(order_date))
       FROM orders
       WHERE order_status = 'Delivered'),
      MAX(DATE(o.order_date))
    ) AS recency,
    COUNT(DISTINCT o.order_id) AS frequency,
    ROUND(
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
      ), 2
    ) AS monetary
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  WHERE o.order_status = 'Delivered'
  GROUP BY o.customer_id
),
rfm_scores AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recency DESC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
```

```
  FROM rfm
)
SELECT
  customer_id,
  recency,
  frequency,
  monetary,
  recency_score,
  frequency_score,
  monetary_score,
  CONCAT(recency_score, frequency_score, monetary_score) AS rfm_score
FROM rfm_scores
ORDER BY monetary DESC;

WITH rfm AS (
  SELECT
    o.customer_id,
    DATEDIFF(
      (SELECT MAX(DATE(order_date))
       FROM orders
       WHERE order_status = 'Delivered'),
      MAX(DATE(o.order_date))
    ) AS recency,
    COUNT(DISTINCT o.order_id) AS frequency,
    ROUND(
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
```

```
        ), 2
      ) AS monetary
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  WHERE o.order_status = 'Delivered'
  GROUP BY o.customer_id
),
rfm_scores AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recency DESC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
  FROM rfm
),
rfm_final AS (
  SELECT *,
    CONCAT(
      recency_score,
      frequency_score,
      monetary_score
    ) AS rfm_score
  FROM rfm_scores
)
SELECT
  customer_id,
  recency,
  frequency,
```

```
    monetary,
    rfm_score,
    CASE
      WHEN rfm_score = '555' THEN 'Champions'
      WHEN recency_score >= 4
        AND frequency_score >= 4 THEN 'Loyal Customers'
      WHEN recency_score >= 4
        AND frequency_score <= 3 THEN 'Potential Loyalists'
      WHEN recency_score <= 2
        AND frequency_score >= 3 THEN 'At Risk'
      ELSE 'Lost Customers'
    END AS customer_segment
FROM rfm_final
ORDER BY monetary DESC;

WITH rfm AS (
    SELECT
      o.customer_id,
      DATEDIFF(
        (SELECT MAX(DATE(order_date))
         FROM orders
         WHERE order_status = 'Delivered'),
        MAX(DATE(o.order_date))
      ) AS recency,
      COUNT(DISTINCT o.order_id) AS frequency,
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
```

```
    ) AS monetary
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  WHERE o.order_status = 'Delivered'
  GROUP BY o.customer_id
),
rfm_scores AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recency DESC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
  FROM rfm
),
rfm_final AS (
  SELECT *,
    CONCAT(recency_score, frequency_score, monetary_score) AS rfm_score
  FROM rfm_scores
)
SELECT
  CASE
    WHEN rfm_score = '555' THEN 'Champions'
    WHEN recency_score >= 4 AND frequency_score >= 4 THEN 'Loyal Customers'
    WHEN recency_score >= 4 AND frequency_score <= 3 THEN 'Potential Loyalists'
    WHEN recency_score <= 2 AND frequency_score >= 3 THEN 'At Risk'
    ELSE 'Lost Customers'
  END AS customer_segment,
  COUNT(*) AS customer_count,
```

```
  ROUND(SUM(monetary), 2) AS segment_revenue
FROM rfm_final
GROUP BY customer_segment
ORDER BY segment_revenue DESC;

SELECT
  cart_status,
  COUNT(*) AS cart_count
FROM cart_activity
GROUP BY cart_status
ORDER BY cart_count DESC;

SELECT
  ROUND(
    SUM(CASE WHEN cart_status = 'Abandoned' THEN 1 ELSE 0 END)
    * 100.0 / COUNT(*),
    2
  ) AS cart_abandonment_rate
FROM cart_activity;

SELECT
  CASE
    WHEN oi.discount_percent = 0 THEN 'No Discount'
    WHEN oi.discount_percent <= 10 THEN '1-10%'
    WHEN oi.discount_percent <= 20 THEN '11-20%'
    WHEN oi.discount_percent <= 30 THEN '21-30%'
    WHEN oi.discount_percent <= 40 THEN '31-40%'
    ELSE '41-50%'
```

```
  END AS discount_range,

  COUNT(DISTINCT o.order_id) AS total_orders,

  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS total_revenue

FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id

WHERE o.order_status = 'Delivered'

GROUP BY discount_range
ORDER BY total_revenue DESC;

SELECT
  payment_method,
  COUNT(*) AS total_transactions,
  ROUND(SUM(payment_amount), 2) AS total_payment_amount
FROM payments
WHERE payment_status = 'Success'
GROUP BY payment_method
ORDER BY total_payment_amount DESC;
```

```
SELECT
  order_status,
  COUNT(*) AS order_count,
  ROUND(
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders),
    2
  ) AS order_percentage
FROM orders
GROUP BY order_status
ORDER BY order_count DESC;


WITH product_sales AS (
  SELECT
    p.product_id,
    p.product_name,
    p.category,
    ROUND(
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
      ), 2
    ) AS total_revenue
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  JOIN products p
    ON oi.product_id = p.product_id
```

```
    WHERE o.order_status = 'Delivered'
    GROUP BY
        p.product_id,
        p.product_name,
        p.category
)
SELECT
    product_id,
    product_name,
    category,
    total_revenue,
    RANK() OVER (
        PARTITION BY category
        ORDER BY total_revenue DESC
    ) AS category_rank
FROM product_sales
ORDER BY category, category_rank;

SELECT
    p.category,

    COUNT(DISTINCT o.order_id) AS total_orders,

    ROUND(
        SUM(
            oi.quantity * oi.unit_price *
            (1 - oi.discount_percent / 100)
        ), 2
```

```
  ) AS total_revenue,

  ROUND(
    SUM(
      oi.quantity *
      (p.selling_price - p.cost_price) *
      (1 - oi.discount_percent / 100)
    ), 2
  ) AS total_profit,

  ROUND(
    SUM(
      oi.quantity * oi.unit_price *
      (1 - oi.discount_percent / 100)
    ) / COUNT(DISTINCT o.order_id),
    2
  ) AS average_order_value

FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id

WHERE o.order_status = 'Delivered'

GROUP BY p.category
ORDER BY total_revenue DESC;
```

```
CREATE TABLE rfm_customer_segments AS

WITH rfm AS (
  SELECT
    o.customer_id,
    DATEDIFF(
      (
        SELECT MAX(DATE(order_date))
        FROM orders
        WHERE order_status = 'Delivered'
      ),
      MAX(DATE(o.order_date))
    ) AS recency,
    COUNT(DISTINCT o.order_id) AS frequency,
    ROUND(
      SUM(
        oi.quantity * oi.unit_price *
        (1 - oi.discount_percent / 100)
      ), 2
    ) AS monetary
  FROM orders o
  JOIN order_items oi
    ON o.order_id = oi.order_id
  WHERE o.order_status = 'Delivered'
  GROUP BY o.customer_id
```

```
),
rfm_scores AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recency DESC) AS recency_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS frequency_score,
    NTILE(5) OVER (ORDER BY monetary ASC) AS monetary_score
  FROM rfm
),
rfm_final AS (
  SELECT *,
    CONCAT(
      recency_score,
      frequency_score,
      monetary_score
    ) AS rfm_score
  FROM rfm_scores
)
SELECT
  customer_id,
  recency,
  frequency,
  monetary,
  recency_score,
  frequency_score,
  monetary_score,
  rfm_score,
  CASE
    WHEN rfm_score = '555' THEN 'Champions'
```

```
      WHEN recency_score >= 4
         AND frequency_score >= 4 THEN 'Loyal Customers'
      WHEN recency_score >= 4
         AND frequency_score <= 3 THEN 'Potential Loyalists'
      WHEN recency_score <= 2
         AND frequency_score >= 3 THEN 'At Risk'
      ELSE 'Lost Customers'
   END AS customer_segment
FROM rfm_final;

SELECT count(*) FROM rfm_customer_segments;



SELECT
   customer_segment,
   COUNT(*) AS customer_count
FROM rfm_customer_segments
GROUP BY customer_segment
ORDER BY customer_count DESC;
```